

Project #2: Concurrent Work Queue Library

Objectives

In this project, you will get practice working with threads, thread pools, and synchronize primitives like mutex locks and condition variables. Your task is to build a library that allows programs to create bounded and concurrent work queues, with accompanying thread pools.

Concurrent Work Queues

Many software systems involve *producers* (components that create work) and *consumers* (components that do or consume what the producers produce) or pools of worker threads that handle tasks as they arrive. In both cases, we often use a work queue to manage the flow of tasks from producers to consumers. In this project, you will write a reusable library that allows test programs to create and use bounded concurrent work queues. Your library will implement the API shown in Listing 1. This API will be defined in a file called **workqueue.h**. I will put the header in the git repository. You should use this header file, as is. **Do not change it, or we may not be able to test your project submission.**

Your library will need to contain the following elements.

- **wq_t:** You will need to define the opaque **wq_t** type. This type represents a work queue created by your library. It should be defined as a struct, and it can contain whatever members you need in order to provide the necessary functionality.
- **wq_create:** This function serves as a constructor for a new work queue with a specified size and an accompanying pool of worker threads. The number of tasks allowed in the queue and the number of worker threads in the associated thread pool are specified by arguments. Once the function has initialized the queue and created the worker threads, the function returns a pointer to a **wq_t** struct. This pointer is passed

Listing 1: `workqueue.h` — your library's interface.

```
#ifndef WORKQUEUE_H
#define WORKQUEUE_H

#include <stdlib.h>
#include <stdbool.h>

typedef struct wq wq_t;
typedef int wq_job_id_t;

typedef void (*wq_job_fn)(void *arg);

extern wq_t *wq_create(size_t num_workers, size_t
    q_capacity);
extern wq_job_id_t wq_submit(wq_t *q, wq_job_fn fn, void
    *arg);
extern void wq_wait(wq_t *q, wq_job_id_t *ids, int numids
    );
extern void wq_shutdown(wq_t *q);

#endif
```

to subsequent functions, in order to tell them what work queue they are operating on—since applications might create many work queues.

- **wq_submit:** An application calls this function to submit a work task to the work queue, where it will wait for an available worker thread to complete it. Each task consists of a pointer to a function (of type **wq_job_fn**—the function that will do the actual work) and a single argument (a **void***). If the queue is full, **wq_submit** blocks (waits) until the task can be added. Once the task is added, the function assigns the task an ID number, and returns that ID back to the calling function. Task IDs should be unique positive integers. You should indicate any errors by returning a **-1**, though any error such case is likely a bug in your code. This function should generally always succeed eventually, assuming that submitted tasks eventually complete. In a real system you would eventually run out of unique task numbers. You can handle this case however you like, but I won't test this functionality (none of my tests will ever create more than 1 billion tasks).
- **wq_wait:** This function takes an array of task IDs and blocks (waits) until they are all finished executing.
- **wq_shutdown:** This function serves as the work queue's destructor. When it is called, all worker threads should complete any tasks they

are currently working on, and then exit. Any tasks still on the queue should be abandoned. All memory allocated for the queue and thread pool should be freed.

- **Worker threads:** A work queue's pool of worker threads should be created in `wq_create` and run until `wq_shutdown` is called. Worker threads should operate in two basic states: working on a task and waiting for a task to arrive on the queue. When a task is added to the queue (with `wq_submit`), the next available worker should grab the task and run it.
- **Other methods:** You may use as many private (static) helper methods as you like within your library to make these required function work properly. Note that these should only exist within your library's source (`.c`) files. You should not add them to the `workqueue.h` file, and they will never be called by any code that is not part of your library.

Building Blocks

For this project you should use the `pthread` library functions to create and synchronize multiple threads. You are not allowed to create extra processes (I don't think that would help anyway), or use prebuilt data structures (`queue.h` is not allowed.) You can call your source files whatever you like, but your compiled library will be called `libworkqueue.a`.

Safety, Efficiency, and Parallelism

Your library functions must be thread safe. I should be able to create multiple work queues of varying thread-pool sizes and queue capacities. I should be able to have multiple threads submitting tasks to the queue concurrently. Worker threads should execute tasks concurrently and when possible (due to multiple cores) in parallel. Using the calls in the API, I should not be able to make your code crash, leak memory, or lose tasks. Mutex locks will come in handy for protecting critical code and preventing race conditions.

Note that you do not need to worry about the thread safety of the submitted tasks. That is the responsibility of the application writer. A race condition in an application task is not your problem (unless it's a task you wrote, in which case you might want to fix it).

In addition to basic functionality, and thread safety, your library will be graded based on efficiency. You want tasks to complete quickly and in parallel when possible, while of course also following the worker and capacity bounds that the application requests.

For example, let's say that I use your library to create a work queue with 6 threads and a queue capacity of 3 tasks. If I then submit 10 IO-bound tasks that each take exactly 1 second to complete, it should take at least 1 second to submit all of the tasks (the last call to `wq_submit` should block for at least 1 second), and it should take a minimum of 2 seconds to complete all of the tasks. If the tasks are completed faster than 2 seconds, then something went wrong (too many workers? too many queue slots?). And, of course, if your library takes 10 seconds to complete the tasks, there's also probably something wrong (unless locks or something else is preventing tasks from running in parallel).

Depending on the application, threads will often do a lot of waiting—waiting for their chance to do something (i.e., workers waiting for a task, a thread waiting for its tasks to be completed). Busy waiting might be a simple way to start out, but it will waste a LOT of CPU cycles and will not get you full points. You should use condition variables instead.

So, your submission will lose points if it is too fast, too slow, or uses too many CPU cycles. Getting full points does not require any fancy compiler optimizations or other heroics. I recommend that you get things working first before you start optimizing things.

1 How to compile your library?

After the last project, you have some experience writing a library, but this time your library will be a static library (called `libworkqueue.a`), meaning that programs will link to it when they are compiled (rather than when they run), and the library will become part of the compiled program. To compile a static library from a set of source files (say, `file1.c` and `file2.c`), you will first compile them to object files (`.o`) and then use the `ar` command to bundle them into a library as shown below:

```
$ gcc -g -Wall -c file1.c file2.c
$ ar -cvrs libworkqueue.a file1.o file2.o
```

You can have more source files, and you can, of course, name them whatever you want; however, it is important that your library is named `libworkqueue.a`. Also, keep in mind that if a `libworkqueue.a` already exists, `ar` will update

that file, replacing the files that you specified. It will not recreate the library from scratch. This has caused students some confusion in the past, and I recommend having your makefile delete the old library and recreate it rather than updating it, each time.

When we test your library we will link our test programs with your library. If I wanted to compile a test, implemented in `workqueuetest1.c`, then I will compile it like so:

```
$ gcc -o executable_name workqueuetest1.c libworkqueue.a
```

Your library must be written in C, and you will need to provide a Makefile that compiles your library. The example above used `gcc`, but you're welcome to use `clang` if you want.

Other thoughts, hints, and warnings

Your program should be written in C and use good programming style (readable code with no magic numbers, good naming and indentation, and appropriate comments). Your code should be `-Wall`¹ clean, and most importantly it should be your own. Your library should also not have any global variables.

Again, I recommend you use `gdb` and `valgrind` to try to track down bugs more efficiently. Keep in mind that concurrency bugs (like race conditions) can be particularly challenging because their behavior can change each time the program is run. Asserts (and other sanity checks) and watchpoints can be particularly helpful in tracking down these bugs.

Make sure that your code compiles, runs and has been thoroughly tested on the lab machines.

Submission Instructions

This project has two deadlines. Absolutely no late assignments will be accepted, and deadline extensions typically require a documented medical emergency or natural disaster.

Compile deadline: 4:00 PM, September 24th — This is a preliminary deliverable worth 10 points. For this, you simply need to submit a valid `tgz` file (more info below on this) with code and a Makefile that compiles without any errors or warnings and produces two files, called **`libworkqueue.a`**. The

¹**“-Wall clean”** = If compiled with the `-Wall` flag, it should not produce any warnings.

compiled program and library do not need to work correctly. For this deliverable, submit to the **compile2** assignment on Canvas.

Functional deadline: 4:00 PM, October 8th — This is the final submission deadline worth the remaining 60 points. For this deliverable, submit to the **project1** assignment on Canvas.

Creating your submission: When your project is complete, archive your source materials, using **tar** (example below):

```
> tar cvzf project2.tgz README Makefile <list of source files>
```

Your **Makefile** should build your program (when I run **make**). It should produce one file, called **libworkqueue.a**.

The **README** file should include your name, a short description of your solution to the project, and any other comments you think are relevant to the grading. It should have two clearly labeled sections titled with KNOWN PROBLEMS, and DESIGN. The KNOWN PROBLEMS section should include a specific description of all known problems or program limitations. This will not hurt your grade and may (in rare cases) improve it. You should also include references to any materials that you referred to while working on the project. Please do not include special instructions for compilation. The compilation and running of the tests will be automated (see details below), and your instructions will not be followed.

Please make sure you include only the source files, not the object files or compiled binaries. Also, make sure that your files are not in a subdirectory within the tar ball. Before you submit this single file, please use the following commands to check and make sure you have everything in the **project2.tgz** file:

```
> mkdir test; cd test
> tar xvzf <path to tgz file>/project2.tgz
> make
```

This should put all of your source files in the current directory, and compile your program and shim.

Submit your **project2.tgz** file via **Canvas**. You must name your archive **project2.tgz**, and it must compile and run on the McAdams Lab machines. We will compile your library using your **Makefile**, and then link it with our test programs.

2 Grading

Your project will be graded based on the results of functional testing and the design of your code. We will run several tests to make sure it works properly and correctly handles various error conditions. Your code should not crash. You will receive 10% credit if your code successfully compiles, and 10% for code style and readability. The rest of your score will be determined by the number of tests that your code passes (60 points) and how well you do on the project quiz (30 points).

Your source materials should be readable, properly documented and use good coding practices (avoid magic numbers, use appropriately-named variables, use good indentation, etc). Your code should be -Wall clean (you should not have any warnings during compilation). **Our testing of your code will be thorough. Be sure you test your library thoroughly.**

Reminders: You can submit to Canvas multiple times before the deadline. Your final submission will be the one that counts even if you have a previous submission with a higher score.

The autograder will run once a day. Starting early means you can get more autograder attempts.

The project quiz is designed to test 1) whether or not you did the project yourself and 2) how well you understand the project and have mastered the technical skills involved in the project. It is designed to be straightforward if you did the project yourself, and rather challenging if you did not. The quiz will be worth 30 points on the project, with one exceptional case. If you score below 5 (out of 30) on the project quiz, you will receive a zero (0) for that project. It will consist of 4–5 questions, and will involve code samples as well as conceptual questions. It will be taken in class on **October 13th** and you will have 20 minutes to complete it.

3 Collaboration/Integrity

You will work independently on this project. You must *not* discuss your solution with classmates, and all of your code must be your own code (no AI-generated, borrowed, or stolen code) produced by your own brain. If the code generation capabilities in your editor are too tempting, consider using something like Vim.

You may, of course, discuss the project with me, and you may discuss conceptual issues related to the project that we have already discussed in lecture

or on Piazza. For example, it would be OK to discuss how **pthread_create** works (in general) since this is something we have discussed in class. It is NOT OK to discuss how you specifically used **pthread_create** to solve any part of this project. **Collaborating with peers on this project, in this or any other manner will be treated as academic misconduct.**

Finally, access to the autograder is a privilege that can be revoked. Please refer to the course syllabus for information about my autograder use policies.